

COMP3320/6464 Week 2, Lecture 3

Rhys Hawkins

Semester 2, 2024

Assignment 1: Molecular Dynamics

The content of these slides is not explicitly examinable

- Methods you learn/use in the assignment are
- There won't be molecular dynamics questions in the exam

`https://gitlab.cecs.anu.edu.au/comp3320/2024/comp3320-2024-assignment-1`

- Fork the repository
- Comprehensive instructions and starting code
- Encourage you to start early
- Especially to make use of the Drop-in Session (week 4) try to complete step 1 before then
- **Due Wednesday, August 28th 5 PM 20% of final grade**

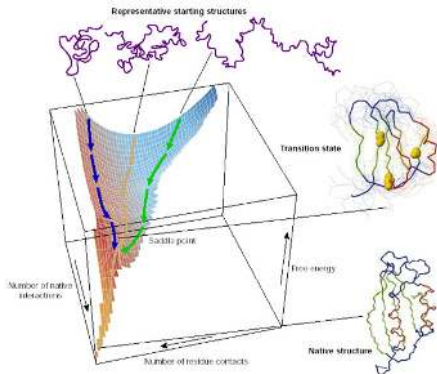
PLAGIARISM

No group work is permitted for any part of the assessment. We do encourage you to discuss your work in the labs and lectures, but we expect you to do the submittable work by yourself.

We do take plagiarism seriously! Please make sure you read the following pages: <https://www.anu.edu.au/students/program-administration/assessments-exams>

Files you have found on the Internet should not be submitted as your work; but your documents may include URL links to external documents. You should avoid copying or closely paraphrasing material from documents that you find, including those of your friends (it's plagiarism) or textbooks. If you feel it necessary to include material from some other document, then it should be clearly identified as such, identifiable as quoted material by layout or quotation marks, and proper attribution made. Be aware that automated tools to detect plagiarism exist and might be used (for example Glatt or JPlag). All assignment submissions will be compared by the course coordinator, with any that are suspiciously similar being investigated. If such similarities cannot be satisfactorily explained, appropriate action (see the penalties mentioned in the handbook) will be taken. Note that the action of providing access to another student to an assignment may result in the offence of assisting plagiarism, and similar penalties (i.e. partial or complete loss of marks) may apply. Don't take such risks!

Assignment 1: Building a MD engine



- to understand molecular behaviour at atomic (and single molecule) detail
- Experimental data are time and ensemble averaged
- Molecular models are generally single structures and static (or cover only short time scales)

Dobson, "Protein folding and misfolding",
Nature 426

Table 9: History and extrapolated future of computer simulations of molecular dynamics. The future is deduced from extrapolation based on an observed increase of computing speed of a factor 10 every 5 years over the past decades (see Figure 31).

Year	Molecular system (type, size)	Length of the simulation [s]
1957	first molecular dynamics simulation (hard discs)	
1964	atomic liquid (argon)	10^{-11}
1971	molecular liquid (water)	5×10^{-12}
1977	protein in a vacuum	2×10^{-11}
1983	protein in water	2×10^{-11}
1989	protein-DNA complex in water	10^{-10}
1997	polypeptide folding in solvent	10^{-7}
2001	micelle formation	10^{-7}
200x	folding of a small protein	10^{-3}

And the future ...

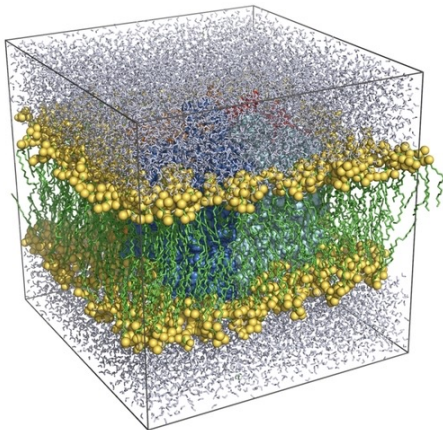
2001	biomolecules in water (ca. 10^4 atoms)	10^{-8}
2029	biomolecules in water (folding sooner?)	10^{-3}
2034	<i>E. coli</i> bacteria (ca. 10^{11} atoms)	10^{-9}
2056	mammalian cell (ca. 10^{15} atoms)	10^{-9}
2080	biomolecules in water (as fast as nature)	10^6
2172	human body (ca. 10^{27} atoms)	1

From: W. F. van Gunsteren et al. *Angew. Chem. Int. Ed.* 2006, 25, 4064 - 4092

Project 1: building an MD engine

You will learn

- How (basic) molecular dynamics simulations work
- How to evaluate performance
- How to optimize performance
- How to visualize simulation data

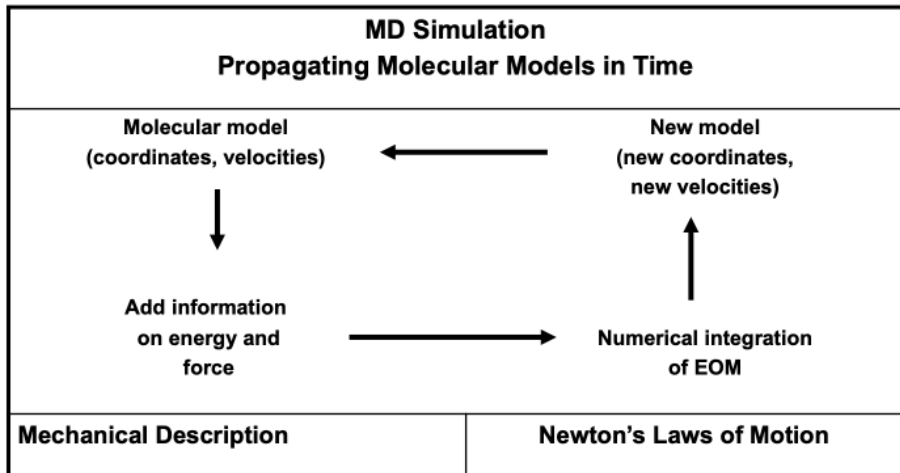


Molecular Dynamics

Approximations we use

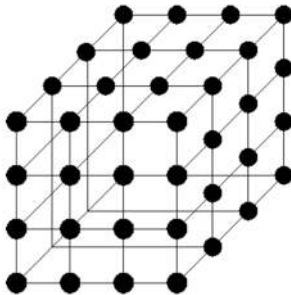
- Classical Mechanics (i.e. no quantum/relativistic effects)
- Atom approximated as point mass with position and velocity
- Newtons equations of motion

Schematic



Initial Atom Positions

Consider now a cubic box with a side length of R . Along each dimension of the box we arrange N atoms as indicated by the following figure (not all atoms are shown!), such that there are a total of $N*N*N$ atoms.

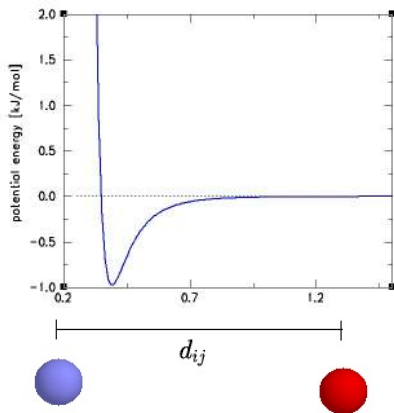


- Velocities and acceleration zero at beginning of simulation

Potential Energy for pairs of atoms

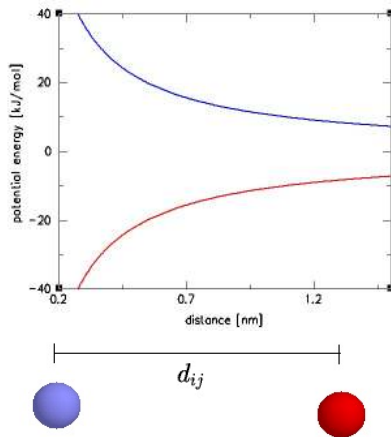
Lennard-Jones

$$V_{\text{LJ}}(\vec{r}_i, \vec{r}_j) = \frac{C_{12}(i, j)}{d_{ij}^{12}} - \frac{C_6(i, j)}{d_{ij}^6}$$



Coulomb

$$V_{\text{Coulomb}}(\vec{r}_i, \vec{r}_j) = \frac{q_i q_j}{4\pi\epsilon\epsilon_0 d_{ij}}$$



Relating forces to energies

- From a scalar potential we can compute force as a vector
- Formally $\mathbf{f} = -\nabla V$
- in 1D: $f_x = -\frac{\partial V}{\partial x}$
- in 3D:

$$\mathbf{f} = \begin{bmatrix} f_x \\ f_y \\ f_z \end{bmatrix} = - \begin{bmatrix} \frac{\partial V}{\partial x} \\ \frac{\partial V}{\partial y} \\ \frac{\partial V}{\partial z} \end{bmatrix}$$

Relating forces to energies

For this assignment we will use

$$V_{ij} = \frac{A}{r_{ij}^{12}} + \frac{B}{r_{ij}^8} + \frac{C}{r_{ij}^4} + \frac{D}{r_{ij}^2}$$

with the values A , B , C , and D given in the assignment description and

$$r_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2} \quad (1)$$

For the x component of the force we compute

$$f_x = -\frac{\partial V_{ij}}{\partial x} = \left(\frac{12A}{r_{ij}^{14}} + \frac{8B}{r_{ij}^{10}} + \frac{4C}{r_{ij}^6} + \frac{2D}{r_{ij}^4} \right) (x_i - x_j)$$

With similar expressions for y and z components.

Newtonian Motion

- Compute forces from potential between each pairwise atom
- From $\mathbf{f} = m\mathbf{a}$ we can compute $\mathbf{a} = \mathbf{f}/m$
- Use finite difference approximation to integrate motion through time
- $a(t) = \frac{d}{dt}v(t) \rightarrow v(t + \Delta t) \sim v(t) + a(t)\Delta t$

$$\mathbf{a}_n = \frac{\mathbf{f}_n}{m}$$

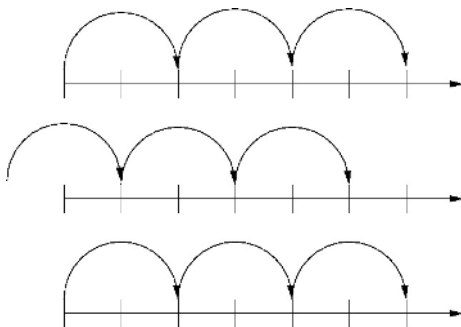
$$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n\Delta t$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_n\Delta t$$

- Accuracy depends on our choice of Δt
- Euler integration performs poorly in practice

A better way to integrate

Leap frog integration (Verlet 1967)



$$\begin{aligned}x(t + \Delta t) &= x(t) + \Delta t v_x(t + \Delta t/2) \\v_x(t + \Delta t/2) &= v_x(t - \Delta t/2) + \Delta t a_x(t) \\a_x(t) &= F_x(t)/m\end{aligned}$$

Velocity Verlet

Since our potential is independent of velocity, we can use a variant called Velocity Verlet

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_n \Delta t + \frac{1}{2} \mathbf{a}_n (\Delta t)^2$$

$$\mathbf{a}_{n+1} = \frac{\mathbf{f}(\mathbf{x}_{n+1})}{m}$$

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \frac{1}{2} (\mathbf{a}_{n+1} + \mathbf{a}_n) \Delta t$$

In your code, each iteration could be written in python as:

```
for i in range(iterations):  
    update_positions(atoms, dt)  
    pe = compute_potentials(atoms)  
    ke = update_velocities(atoms, dt)
```

Monitoring Numerical Dispersion

Total Energy Should Be Conserved

$$PE = \sum V_{ij}$$

$$KE = \sum \frac{1}{2} m |\mathbf{v}|^2$$

In the previous psuedo code `pe` and `ke` should be computed in order to track total energy through time, i.e. `pe + ke`.

Reasons why this may not occur

- Bug
- Timestep too large
- `pe` should be negative
- Gradual numerical error accumulation (unavoidable)

Project Outline

Steps (More Detail in Gitlab Repo)

- Potential interaction energy
- Functional prototype code in python to integrate equations of motion
- Two basic versions in C
- Analysis/Benchmarking
- Final Optimised code
- Report

Example Output

```
> python3 md.py 3 1 0.001 10
Initial grid spacing : 1.000000
Total number of atoms : 27
Integration time step : 0.001000
Number of timesteps : 10
```

TIME	Total	Kinetic	Potential
0.000000	-7.577011e+01	0.000000e+00	-7.577011e+01
0.001000	-7.577011e+01	1.504815e-04	-7.577026e+01
0.002000	-7.577011e+01	6.018923e-04	-7.577071e+01
0.003000	-7.577011e+01	1.354131e-03	-7.577147e+01
0.004000	-7.577011e+01	2.407030e-03	-7.577252e+01
0.005000	-7.577011e+01	3.760354e-03	-7.577387e+01
0.006000	-7.577011e+01	5.413798e-03	-7.577552e+01
0.007000	-7.577011e+01	7.366993e-03	-7.577748e+01
0.008000	-7.577011e+01	9.619501e-03	-7.577973e+01
0.009000	-7.577011e+01	1.217082e-02	-7.578228e+01

- There is a useful script in the repository that tests each implementation of the code for correctness
- `python auto_test.py`

Summary

- **Due Tuesday, August 28th 5 PM 20% of final grade**
- First stage: prototype with python
- Second stage: analysis of AoS and SoA (lectures next week useful)
- Third stage: Optimized version (subsequent lectures)
- Marking of the assignment is based on analysis and report, not necessarily the best optimized